

Based on the server code you shared, the answer is **definitely no**.

Your server only updates variables that satisfy:

if `node_class == "Variable"` and `is_runtime_variable(browse_name):`

and `is_runtime_variable()` only matches names containing:

temperature
pressure
torque
angle
rotation
speed
status
fault
error
result
measuredvalue
highlimit
lowlimit
workorder
task

Therefore:

Variables that WILL get random values

Examples:

Temperature
Health.Temperature
Pressure
Torque
MeasuredValue
RotationSpeed
Status
FaultCode
TaskId
WorkOrder

These will be found and updated every second.

Variables that will NOT get random values

Many variables in your sample XML such as:

NamespaceUri
NamespaceVersion
SerialNumber
Capacity
NominalVoltage
EngineeringUnits
InitialOperationDate
StaticNodeIdTypes
StaticNumericNodeIdRange
DefaultRolePermissions
Description
DisplayName

will never be added to `runtime_variables`.

Therefore they remain:

Imported value
or
Default value
or
NodeSet value

depending on how python-opcua imported them.

Another important point

Your server is not checking:

all UAVariables in XML

It is checking:

all UAVariables whose BrowseName contains specific keywords

These are very different things.

How to verify

After startup, print:

```
logging.info(  
    "Runtime variables found: %d",  
    len(runtime_variables)  
)
```

Now compare that number against the number of UAVariables in your XML.

For example:

XML UAVariables = 2500
Runtime variables found = 45

Then only 45 variables are being randomized.

If your goal is

SIOME5 XML template



Import to OPC UA server



Every primitive UAVariable gets changing data



Exporter generates XML

then I would recommend:

```
if node_class == "Variable":  
    runtime_variables[node_id_str] = (browse_name, child)
```

instead of:

```
if node_class == "Variable" and is_runtime_variable(browse_name):
```

and then generate values based on datatype:

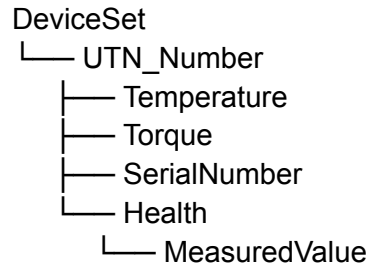
Boolean -> True/False
Double -> random float
Int32 -> random int
String -> random string
DateTime -> current time

That would cover **all primitive variables in the imported NodeSet**, not just the few keyword-based ones.

Think of OPC UA as a database of machine information

Every OPC UA server exposes an **Information Model**.

For example, your SIOME5 server might expose:



Each item has:

NodeId
BrowseName
DataType
Value
References

For example:

NodeId : ns=6;s=Objects.DeviceSet.UTN_Number.Health.Temperature
BrowseName : Temperature
DataType : Double
Value : 72.5

What your SIOME5 exporter currently expects

Your exporter is written assuming the OPC UA server contains nodes like:

ns=6;s=Objects.DeviceSet.UTN_Number.Health.Temperature
ns=6;s=Objects.DeviceSet.UTN_Number.Health.Pressure
ns=6;s=Objects.DeviceSet.UTN_Number.AssetManagement...

because those NodeIds come from the SIOME5 template.

So exporter does:

Template XML



Find NodeId in XML



Read same NodeId from OPC UA server



Replace value

Example:

Template contains:

```
<UAVariable  
  NodeId="ns=6;s=Objects.DeviceSet.UTN_Number.Health.Temperature">
```

Exporter asks server:

```
client.get_node(  
  "ns=6;s=Objects.DeviceSet.UTN_Number.Health.Temperature"  
)
```

and gets:

72.5

then writes:

```
<uax:Double>72.5</uax:Double>
```

What happens with a UR10e server?

A UR10e server has its own Information Model.

It may look like:

Robot

- |— Joint1
- |— Joint2
- |— Joint3
- |— Joint4
- |— Joint5
- |— Joint6
- |— TCP_X
- |— TCP_Y

└─ TCP_Z
└─ RobotMode

with NodeIds such as:

ns=2;s=Robot.Joint1
ns=2;s=Robot.Joint2
ns=2;s=Robot.TCP_X

Notice:

SIOME5 NodeIds

≠

UR10e NodeIds

They are completely different.

Why exporter fails

Exporter asks:

```
client.get_node(  
"ns=6;s=Objects.DeviceSet.UTN_Number.Health.Temperature"  
)
```

UR10e server says:

Node not found

because it only contains:

ns=2;s=Robot.Joint1
ns=2;s=Robot.Joint2
...

not:

Objects.DeviceSet.UTN_Number...

What a mapping does

A mapping is simply a translation table.

You tell exporter:

SIOME5 field



take value from this UR10e node

Example:

```
MAPPING = {  
  "Temperature":  
    "ns=2;s=Robot.ControllerTemperature",  
  
  "MeasuredValue":  
    "ns=2;s=Robot.TCP_X",  
  
  "Status":  
    "ns=2;s=Robot.RobotMode"  
}
```

Then:

SIOME5 XML field

Temperature



read

ns=2;s=Robot.ControllerTemperature



write value into XML

Visual example

Without mapping

SIOME5 XML

Temperature



Read same node from UR10e



Node doesn't exist



No value

With mapping

SIOME5 XML

Temperature



Mapping table



Robot.ControllerTemperature



36.2



Write 36.2 into XML

Result:

```
<UAVariable BrowseName="Temperature">
```

```
  <Value>
```

```
    <uax:Double>36.2</uax:Double>
```

```
  </Value>
```

```
</UAVariable>
```

Why industries do this

This is actually very common.

Suppose you have:

- Fanuc robots
- UR robots
- Atlas Copco tools
- Siemens PLCs

All expose different OPC UA Information Models.

A manufacturing platform often wants:

One standard format

such as:

SIOME5

AutomationML

ISA-95
Custom MES schema

Therefore:

Vendor OPC UA Model
↓
Mapping Layer
↓
Common Enterprise Model

In your project

Your current architecture is closer to:

SIOME5 XML Template
↓
SIOME5-compatible OPC UA Server
↓
Exporter
↓
SIOME5 XML

Since your simulated server was created from the same NodeSet XML, **no mapping is needed** because:

Template NodeIds
=
Server NodeIds

This is why your current setup works well.

If tomorrow you replace the server with:

- UR10e
- Fanuc
- Siemens PLC
- Beckhoff

then you'll need either:

1. **Mapping layer** (most common in industry), or
2. A completely new exporter for that server's information model.

For your SIOME5 use case, building the simulated server from the SIOME5 NodeSet was actually a good design choice because it avoids the mapping problem entirely.

but in case of ur10 opcua server NodeId :

ns=6;s=Objects.DeviceSet.UTN_Number.Health.Temperature BrowseName : Temperature
DataType : Double Value : 72.5 these will not be same right? meaning node id, browsename ,
namespace and weill be diffrent right

Exactly. In a UR10e OPC UA server, those things will typically be different.

Think of it this way:

Your SIOME5-based server

Because you imported the SIOME5 NodeSet:

Namespace URI:

<http://airbus.com/IJT/AFastening>

NodeId:

ns=6;s=Objects.DeviceSet.UTN_Number.Health.Temperature

BrowseName:

Temperature

DataType:

Double

The server exposes the same information model as the SIOME5 template.

A UR10e OPC UA server

Universal Robots designed their own information model.

It may look more like:

Namespace URI:

urn:UniversalRobots:UR10e

NodeId:

ns=2;s=Robot.Joints.Joint1.Position

BrowseName:

Position

DataType:
Double

or

NodeId:
ns=2;s=Robot.TCP.X

BrowseName:
X

DataType:
Double

The exact names depend on the UR OPC UA implementation, but the principle is the same.

What usually changes?

Attribute	SIOME5 Server	UR10e Server
Namespace URI	http://airbus.com/IJT/AFas_tening	UR-specific URI
Namespace Index	ns=6	Often ns=2, ns=3, etc.
NodeId	SIOME5 hierarchy	UR hierarchy
BrowseName	SIOME5 names	UR names
Parent-child structure	SIOME5 model	UR model
References	SIOME5 references	UR references
DataType	May be same	Often same for measurements

What might stay the same?

Some physical concepts are common:

Temperature
Position
Velocity
Current
Voltage
Status

But even these are often named differently.

Example:

SIOME5

Health.Temperature

UR10e

ControllerTemperature

or

RobotTemperature

Both mean temperature, but the NodeIds and hierarchy differ.

Why your simulated server is special

Your simulated server was built from:

AtlasCopco-Tools.Nodeset2.xml

using:

```
server.import_xml(...)
```

So the server's information model becomes:

SIOME5 XML

↓

OPC UA Address Space

This means:

Template XML NodeId

=

Server NodeId

Template BrowseName

=

Server BrowseName

Template hierarchy

=

Server hierarchy

which is why your exporter can simply do:

```
client.get_node(server_node_id)
```

and find everything.

A useful analogy

Imagine SIOME5 is a spreadsheet:

Column A: Temperature

Column B: Torque

Column C: Status

Your simulated server uses the exact same spreadsheet.

A UR10e server might use:

Column A: ControllerTemp

Column B: JointTorque

Column C: RobotMode

The information is there, but the column names are different.

That's why a mapping layer is needed when exporting UR10e data into a SIOME5 XML format.

So your understanding is correct:

For a UR10e OPC UA server, the Namespace URI, Namespace Index, NodeIds, BrowseNames, hierarchy, and references will generally be different from the SIOME5 template. Only the underlying engineering meaning (temperature, position, status, etc.) may be similar.

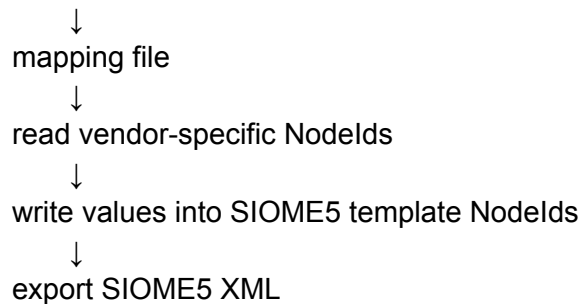
now how can i create a exporter code which can work with different opcua server but following the same siome5 format tree strcuture?

The clean way is to separate two responsibilities: keep SIOME5 XML as the fixed output structure, and add a mapping layer that knows where each value comes from in each vendor OPC UA server.

Thought for a couple of seconds

You need a **template + mapping based exporter**.

Any OPC UA server



Your SIOME5 tree should still come from the template, not from the UR10e/Fanuc/PLC server.

1. Keep SIOME5 template as output structure

Do **not** generate structure from UR10e server.

Use:

AtlasCopco-Tools.Nodeset2.xml

as the fixed output blueprint.

This guarantees:

SIOME5 NodeIds
SIOME5 BrowseNames
SIOME5 References
SIOME5 metadata
SIOME5 tree structure

remain same as expected by TIA Portal. Your current exporter already follows this idea.

2. Add a mapping file

Example `mapping_ur10e_to_siome5.json`:

```
{
  "ns=6;s=Objects.DeviceSet.UTN_Number.Health.Temperature": {
    "source_node_id": "ns=2;s=Robot.ControllerTemperature",
    "data_type": "Double"
  },
  "ns=6;s=Objects.DeviceSet.UTN_Number.Status": {
    "source_node_id": "ns=2;s=Robot.RobotMode",
    "data_type": "String"
  },
  "ns=6;s=Objects.DeviceSet.UTN_Number.TCP.X": {
    "source_node_id": "ns=2;s=Robot.TCP.X",
    "data_type": "Double"
  }
}
```

Meaning:

SIOME5 target node



take value from this UR10e source node

3. Exporter logic

For every `UAVariable` in SIOME5 template:

Check SIOME5 NodeId



Is it in mapping?



Yes → read mapped OPC UA node from UR10e/Fanuc/PLC server



No → keep template value unchanged

4. Simplified code pattern

```
import json
from lxml import etree
from opcua import Client
```

```
UA_NS = "http://opcfoundation.org/UA/2011/03/UANodeSet.xsd"
```

```
UAX_NS = "http://opcfoundation.org/UA/2008/02/Types.xsd"
```

```
def export_siome5_with_mapping(endpoint_url, template_path, mapping_path):  
    with open(mapping_path, "r", encoding="utf-8") as f:  
        mapping = json.load(f)
```

```
    client = Client(endpoint_url)  
    client.connect()
```

```
    try:
```

```
        tree = etree.parse(template_path)  
        root = tree.getroot()
```

```
        for var in root.findall(f"{{{UA_NS}}}UAVariable"):  
            siome5_node_id = var.get("NodeId")
```

```
            if siome5_node_id not in mapping:  
                continue
```

```
            source_node_id = mapping[siome5_node_id]["source_node_id"]
```

```
            try:
```

```
                live_value = client.get_node(source_node_id).get_value()  
            except Exception:  
                continue
```

```
            update_value(var, live_value)
```

```
    return etree.tostring(  
        root,  
        encoding="utf-8",  
        xml_declaration=True,  
        pretty_print=True  
    )
```

```
    finally:
```

```
        client.disconnect()
```

```
def update_value(var_node, live_value):
```

```
    value_node = var_node.find(f"{{{UA_NS}}}Value")
```

```

if value_node is not None:
    var_node.remove(value_node)

value_node = etree.SubElement(var_node, f"{{{UA_NS}}}Value")

data_type = var_node.get("DataType", "String")

if "Double" in data_type:
    child = etree.SubElement(value_node, f"{{{UAX_NS}}}Double")
    child.text = str(float(live_value))

elif "Boolean" in data_type:
    child = etree.SubElement(value_node, f"{{{UAX_NS}}}Boolean")
    child.text = str(bool(live_value)).lower()

elif "DateTime" in data_type:
    child = etree.SubElement(value_node, f"{{{UAX_NS}}}DateTime")
    child.text = str(live_value)

else:
    child = etree.SubElement(value_node, f"{{{UAX_NS}}}String")
    child.text = str(live_value)

```

Final architecture

SIOME5 Template XML



Exporter reads each SIOME5 UVariable



Mapping decides source OPC UA node



UR10e / Fanuc / PLC live value is read



Value inserted into SIOME5 XML



Final SIOME5-compatible XML

So the exporter can support many servers, but only if you provide a **server-specific mapping file**:

mapping_ur10e_to_siome5.json

mapping_fanuc_to_siome5.json

mapping_siemens_plc_to_siome5.json